Bisección y recorridos en amplitud y en profundidad: soluciones

Programación: UVa 12032, UVa 336 y UVa 11686

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

Contenido

1 Bisección: encontrar la función

2 Recorrido en amplitud con tope de pasos

3 Recorrido en profundidad y ciclos

Contenido

1 Bisección: encontrar la función
- Solución: UVa 12032

2 Recorrido en amplitud con tope de pasos

3 Recorrido en profundidad y ciclos

La función y sus límites

La decisión

$\varphi(k)$: se simula la subida llevando la fuerza actual. Cada salto se compara contra ella, y el que la iguala la gasta en uno. Una pasada, $\Theta(n)$.

Por qué es monótona

Con $k+1$ la fuerza disponible es siempre mayor o igual que con k, así que ningún salto que cabía deja de caber.

Los límites

Por debajo, el salto más grande, que hay que dar. Por arriba, ese mismo más n: cada salto baja la fuerza a lo sumo en uno.

La decisión, en Python

```python
def alcanza(saltos, k):
    # Con fuerza inicial k, ¿se llega al último peldaño?
    fuerza = k
    puede = True
    for s in saltos:
        if s > fuerza:
            puede = False
        elif s == fuerza:
            fuerza = fuerza - 1
    return puede
```

El detalle

El ciclo no se corta cuando falla: deja la bandera en falso y sigue. Es lo mismo y respeta el único `return`.

La bisección, en Python

```
def minimaFuerza(saltos):
    # El menor k que cumple alcanza(saltos, k).
    lo = max(saltos)
    hi = lo + len(saltos)
    while lo < hi:
        medio = (lo + hi) // 2
        if alcanza(saltos, medio):
            hi = medio
        else:
            lo = medio + 1
    return lo
```

El intervalo

`lo` es el primer candidato que podría servir y `hi` el primero que seguro sirve. Cuando se juntan, ahí está el borde.

La misma decisión, en C++

```cpp
bool alcanza(const vector<int> &saltos, int k) {
  // Con fuerza inicial k, se llega al ultimo peldano?
  int fuerza = k;
  bool puede = true;
  for (size_t i = 0; i < saltos.size(); i++) {
    if (saltos[i] > fuerza) {
      puede = false;
    } else if (saltos[i] == fuerza) {
      fuerza = fuerza - 1;
    }
  }
  return puede;
}
```

La misma bisección, en C++

```cpp
int minimaFuerza(const vector<int> &saltos) {
  // El menor k que cumple alcanza(saltos, k).
  int lo = 0;
  for (size_t i = 0; i < saltos.size(); i++) {
    if (saltos[i] > lo) {
      lo = saltos[i];
    }
  }
  int hi = lo + (int)saltos.size();
  while (lo < hi) {
    int medio = lo + (hi - lo) / 2;
    if (alcanza(saltos, medio)) {
      hi = medio;
    } else {
      lo = medio + 1;
    }
  }
  return lo;
}
```

Dos detalles de la versión en C++

El punto medio

Se escribe `lo + (hi - lo) / 2` y no `(lo + hi) / 2`: con valores grandes la suma se desborda y la resta no.

El paso por referencia

Los saltos van como `const vector<int> &`: sin el `&` se copiaría el arreglo entero en cada una de las vueltas de la bisección.

Contenido

1 Bisección: encontrar la función

2 Recorrido en amplitud con tope de pasos
- Solución: UVa 336

3 Recorrido en profundidad y ciclos

La idea

Un recorrido por consulta

Se arma la lista de adyacencia, con un diccionario que traduce el número de cada nodo a un índice. Cada consulta corre un recorrido con una cola y un arreglo de distancias que arranca en -1. La respuesta es el total de nodos menos los alcanzados, y el nodo de partida cuenta como alcanzado, incluso con TTL 0.

El diccionario, en Python

```
def indiceDe(indice, adj, nodo):
    # El índice del nodo; si es la primera vez que aparece, se le asigna uno.
    if nodo not in indice:
        indice[nodo] = len(indice)
        adj.append([])
    return indice[nodo]
```

Índices seguidos

Cada nodo nuevo recibe como índice el tamaño del diccionario y añade su lista vacía a `adj`. Los índices quedan entre 0 y $V - 1$.

El recorrido, en Python

```
def bfs(adj, inicio, ttl):
    # Cuántos nodos quedan a distancia menor o igual a ttl desde inicio.
    dist = [-1] * len(adj)
    cola = deque()
    dist[inicio] = 0
    cola.append(inicio)
    alcanzados = 1
    while len(cola) > 0:
        u = cola.popleft()
        if dist[u] < ttl:
            for v in adj[u]:
                if dist[v] == -1:
                    dist[v] = dist[u] + 1
                    alcanzados = alcanzados + 1
                    cola.append(v)
    return alcanzados
```

Lo que llena la lectura, en Python

```
ia = indiceDe(indice, adj, int(datos[pos]))
ib = indiceDe(indice, adj, int(datos[pos + 1]))
pos = pos + 2
adj[ia].append(ib)
adj[ib].append(ia)
```

```
caso = caso + 1
alcanzados = 0
if nodo in indice:
    alcanzados = bfs(adj, indice[nodo], ttl)
salida.append('Case ' + str(caso) + ': ' + str(len(adj) - alcanzados)
              + ' nodes not reachable from node ' + str(nodo)
              + ' with TTL = ' + str(ttl) + '.')
```

Antes de llamar

La primera parte arma la lista y la segunda responde una consulta. Si el nodo no está en la red, los alcanzados son cero.

El diccionario, en C++

```
int indiceDe(map<int, int> &indice, vector<vector<int> > &adj, int nodo) {
  // El indice del nodo; si es la primera vez que aparece, se le asigna uno.
  if (indice.count(nodo) == 0) {
    int nuevo = (int)indice.size();
    indice[nodo] = nuevo;
    adj.push_back(vector<int>());
  }
  return indice[nodo];
}
```

El recorrido, en C++

```cpp
int bfs(const vector<vector<int> > &adj, int inicio, int ttl) {
  // Cuantos nodos quedan a distancia menor o igual a ttl desde inicio.
  vector<int> dist(adj.size(), -1);
  queue<int> cola;
  dist[inicio] = 0;
  cola.push(inicio);
  int alcanzados = 1;
  while (!cola.empty()) {
    int u = cola.front();
    cola.pop();
    if (dist[u] < ttl) {
      for (size_t i = 0; i < adj[u].size(); i++) {
        int v = adj[u][i];
        if (dist[v] == -1) {
          dist[v] = dist[u] + 1;
          alcanzados = alcanzados + 1;
          cola.push(v);
        }
      }
    }
  }
  return alcanzados;
}
```

Lo que llena la lectura, en C++

```
int ia = indiceDe(indice, adj, a);
int ib = indiceDe(indice, adj, b);
adj[ia].push_back(ib);
adj[ib].push_back(ia);
```

```
caso = caso + 1;
int alcanzados = 0;
if (indice.count(nodo) > 0) {
  alcanzados = bfs(adj, indice[nodo], ttl);
}
printf("Case %d: %d nodes not reachable from node %d with TTL = %d.\n",
       caso, (int)adj.size() - alcanzados, nodo, ttl);
```

Antes de consultar

Sin el `count`, `indice[nodo]` crea la clave y lee fuera de `adj`, sin que el programa falle.

Las trampas

Qué sale mal

- Con `dist[u] <= ttl`, la muestra da 1, 0, 5, 0, 1, 0 en lugar de 5, 1, 8, 5, 3, 1.
- Reiniciar el contador en cada red numera la muestra 1, 2, 1, 2, 3, 4.
- El enunciado no descarta un nodo ausente: la solución responde con el total de la red.

Lo que cuesta

Por consulta

El recorrido cuesta $O(V + E)$, con $V \leq 30$ y $E \leq 435$. Cada búsqueda en el diccionario cuesta $O(\log V)$ en C++ y $O(1)$ en promedio en Python.

Contenido

1 Bisección: encontrar la función

2 Recorrido en amplitud con tope de pasos

3 Recorrido en profundidad y ciclos
- Solución: UVa 11686

La idea

Tres estados y el orden invertido

Se llama a `dfs` desde todo nodo sin visitar. Una arista a un nodo en camino cierra un ciclo, y la respuesta es `IMPOSSIBLE`. Si no apareció ninguno, cada nodo terminó después de todo lo que sale de él, de modo que el orden de terminación invertido respeta todas las aristas.

La función, en Python

```
def dfs(adj, color, orden, u):
    # Recorre lo que sale de u y devuelve si encontró un ciclo.
    hayCiclo = False
    color[u] = EN_CAMINO
    for v in adj[u]:
        if color[v] == EN_CAMINO:
            hayCiclo = True
        elif color[v] == SIN_VISITAR:
            if dfs(adj, color, orden, v):
                hayCiclo = True
    color[u] = TERMINADO
    orden.append(u)
    return hayCiclo
```

Lo que llena la lectura, en Python

```
color = [SIN_VISITAR] * (n + 1)
orden = []
hayCiclo = False
u = 1
while u <= n:
    if color[u] == SIN_VISITAR:
        if dfs(adj, color, orden, u):
            hayCiclo = True
    u = u + 1
if hayCiclo:
    salida.append('IMPOSSIBLE')
else:
    orden.reverse()
    salida.append('\n'.join([str(u) for u in orden]))
```

Un solo punto de salida

La bandera se enciende y el recorrido sigue hasta el final.

La función, en C++

```cpp
void dfs(const vector<vector<int> > &adj, vector<int> &color,
         vector<int> &orden, bool &hayCiclo, int u) {
  // Recorre lo que sale de u; una arista a un nodo en camino cierra un ciclo.
  color[u] = EN_CAMINO;
  for (size_t i = 0; i < adj[u].size(); i++) {
    int v = adj[u][i];
    if (color[v] == EN_CAMINO) {
      hayCiclo = true;
    } else if (color[v] == SIN_VISITAR) {
      dfs(adj, color, orden, hayCiclo, v);
    }
  }
  color[u] = TERMINADO;
  orden.push_back(u);
}
```

Lo que llena la lectura, en C++

```
vector<int> color(n + 1, SIN_VISITAR);
vector<int> orden;
bool hayCiclo = false;
for (int u = 1; u <= n; u++) {
  if (color[u] == SIN_VISITAR) {
    dfs(adj, color, orden, hayCiclo, u);
  }
}
if (hayCiclo) {
  printf("IMPOSSIBLE\n");
} else {
  for (int i = n - 1; i >= 0; i--) {
    printf("%d\n", orden[i]);
  }
}
```

La recursiva revienta

Medido con una pila de 8 MB y `-O2`

La recursiva de C++ pasó con 86 851 niveles y reventó con 87 826, unos 87 mil. El enunciado admite un millón de palitos en cadena, de modo que se envía la versión con pila, que guarda en un `vector` lo mismo que guardaban las llamadas.

La versión con pila: qué guarda

Dos cosas de cada llamada

- El `vector pila` es el camino actual: su tope es el nodo que corre.
- `siguiente[u]` recuerda cuál arista de u toca revisar, el dato que en la recursiva guarda la variable `i`.

Mientras le queden aristas, el tope mira una. Sin aristas, termina y sale de la pila.

La versión con pila, en C++

```
void dfs(const vector<vector<int> > &adj, vector<int> &color,
  vector<size_t> &siguiente, vector<int> &orden, bool &hayCiclo, int inicio) {
  vector<int> pila;
  color[inicio] = EN_CAMINO;
  pila.push_back(inicio);
  while (!pila.empty()) {
    int u = pila.back();
    if (siguiente[u] < adj[u].size()) {
      int v = adj[u][siguiente[u]];
      siguiente[u] = siguiente[u] + 1;
      if (color[v] == EN_CAMINO) {
        hayCiclo = true;
      } else if (color[v] == SIN_VISITAR) {
        color[v] = EN_CAMINO;
        pila.push_back(v);
      }
    } else {
      color[u] = TERMINADO;
      orden.push_back(u);
      pila.pop_back();
    }
  }
}
```

La versión con pila: dónde se crea `siguiente`

Una vez por caso

`siguiente` se crea en `main` y no dentro de `dfs`. Dentro de la función cuesta $\Theta(n)$ por raíz: en una entrada con muchas raíces, y sin `-O2`, esa variante seguía corriendo tras 305 s, frente a 1,07 s de esta.

Cuál enviar al juez

Programa	Cadena de un millón	Memoria
C++ recursiva	revienta a los 0,46 s	
C++ con pila	0,24 s	80 MB
Python 3.14	1,43 s	482 MB
Python 3.8	revienta a los 1,74 s	

Se envía la de C++ con pila

El juez corre una versión vieja de Python 3, y Python 3.8 revienta con 10 968 niveles, con el mismo límite de recursión.

Las trampas

Qué sale mal

- Con dos estados, el diamante y una arista repetida salen `IMPOSSIBLE`.
- Sin invertir el orden, la muestra da 3, 2, 1 en lugar de 1, 2, 3.
- Arrancar solo desde el palito 1 deja nodos por fuera.
- Un par a a es un ciclo de longitud uno.

Lo que cuesta

Por caso

$\Theta(n + m)$ en las tres soluciones, porque cada nodo se visita una vez y cada arista se mira una vez. La memoria adicional del recorrido es $\Theta(n)$, en la pila de llamadas o en el `vector`.